

Session 10 – Supply chain et sécurité

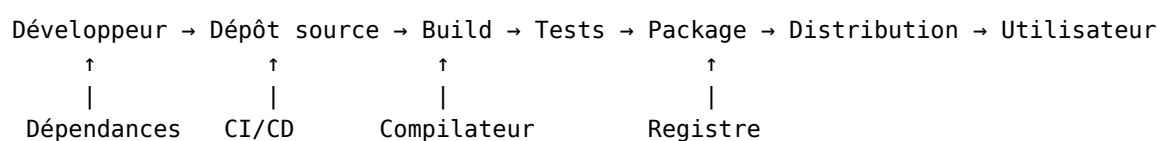
Objectifs de la séance

- Comprendre ce qu'est une *supply chain* logicielle.
- Identifier les risques de sécurité spécifiques à l'open source.
- Expliquer ce qu'est un **SBOM** et son importance.
- Analyser les incidents majeurs (**Log4Shell**, **XZ Utils**).
- Positionner les obligations du **CRA** (fabricant, distributeur, *steward*).
- Appréhender l'impact des **LLM** sur la sécurité open source.

Partie 1 — La supply chain logicielle

Définition

Ensemble des composants, outils, processus et personnes impliqués dans le développement et la distribution d'un logiciel.



La réalité des dépendances

Dépendances directes : les bibliothèques que vous importez explicitement, les frameworks utilisés, les outils de build.

Dépendances transitives : les dépendances de vos dépendances. Souvent 10× plus nombreuses, moins visibles et plus risquées.

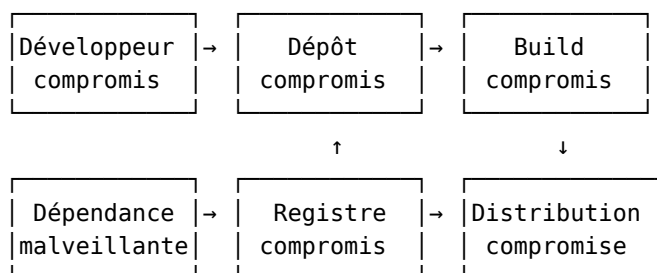
Exemple : une application React simple peut avoir 500+ dépendances npm.

Statistiques

Écosystème	Packages	Dépendances moyennes
npm	2+ M	5-10 directes, 50-100 transitives
PyPI	500 k+	3-5 directes, 20-50 transitives
Maven	500 k+	10-20 directes, 100+ transitives
Cargo	120 k+	Variable

Plus de 90 % du code d'une application moderne provient de dépendances open source.

Points d'attaque



Partie 2 — Incidents majeurs

Log4Shell (décembre 2021)

CVE-2021-44228, score CVSS 10.0 (critique).

La vulnérabilité : Log4j, bibliothèque de logging Java utilisée par des millions d'applications. Une injection JNDI permet l'exécution de code à distance.

L'impact : exploitation triviale (une simple chaîne dans n'importe quel champ loggé) ; découverte le 24 novembre 2021 ; divulgation publique le 9 décembre 2021 ; chaos mondial pendant des semaines.

Leçons :

1. **Invisibilité des dépendances** — beaucoup ne savaient pas utiliser Log4j (dépendance transitive).
2. **Mainteneur unique** — projet critique, quelques bénévoles.
3. **Temps de réponse** — difficulté à identifier et patcher tous les systèmes.
4. **Dépendances transitives** — Log4j était souvent indirect.

XZ Utils (mars 2024)

La porte dérobée la plus sophistiquée jamais découverte dans l'open source.

Chronologie : 2021, « Jia Tan » commence à contribuer ; 2022-2023, il gagne la confiance ; 2024, il devient co-mainteneur ; mars 2024, la backdoor est découverte.

Sophistication de l'attaque : ingénierie sociale étalée sur 3 ans visant le mainteneur surchargé ; code obfusqué caché dans les fichiers de tests ; cible SSH sur les systèmes Debian/Fedora ; découverte par hasard — un ingénieur Microsoft (Andres Freund) repère une latence anormale de 500 ms.

Leçons :

1. **Social engineering** — l'attaquant a manipulé le mainteneur surchargé (burnout).
2. **Single maintainer** — un seul mainteneur pour une lib critique.
3. **Confiance aveugle** — les distributions incluent automatiquement.
4. **Détection difficile** — découvert par un ingénieur Microsoft (Andres Freund) par chance.

■ « *We're all just one mass psychosis away from disaster.* »

Autres incidents notables

Incident	Année	Type	Impact
Heartbleed (OpenSSL)	2014	Buffer over-read	Fuite de mémoire serveur, clés TLS
leftpad	2016	Suppression d'un paquet npm de 11 lignes	Milliers de builds cassés
event-stream	2018	Mainteneur malveillant	Vol de wallets Bitcoin (Copay)
ua-parser-js	2021	Compte npm compromis	Cryptominers
colors / faker	2022	Sabotage par l'auteur	Boucle infinie, protestation non-rémunération
node-ipc	2022	<i>Protestware</i> (guerre Ukraine)	Fichiers écrasés sur IP russes/bielorusses
Shai Hulud	2025	Ver auto-propageant npm	Exfiltration de tokens
PyPI typosquatting	continu	Faux packages	Malware divers

Partie 3 — SBOM et inventaire

Qu'est-ce qu'un SBOM ?

Software Bill of Materials — la liste des ingrédients d'un logiciel.

Ce que contient un SBOM : tous les composants utilisés, leurs versions, leurs licences, leurs dépendances, leur provenance.

Formats standards : **SPDX** (Linux Foundation, normalisé ISO/IEC 5962:2021), **CycloneDX** (projet OWASP, normalisé ECMA-424), **SWID** (ISO/IEC).

Exemple (CycloneDX)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "components": [{
    "type": "library",
    "name": "log4j-core",
    "version": "2.14.1",
    "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",
    "licenses": [{"license": {"id": "Apache-2.0"}}]
  }]
}
```

Pourquoi le SBOM est devenu incontournable

- **Executive Order 14028** (USA, mai 2021) : les fournisseurs de logiciels au gouvernement fédéral doivent fournir un SBOM.
- **Cyber Resilience Act** (UE, publié nov. 2024, applicable **11 déc. 2027**) : premier règlement européen encadrant les produits numériques, **y compris l'open source**. 36 mois d'adaptation.

SBOM = Sécurité + Conformité + Soutenabilité :

1. **Sécurité** — détection d'impact immédiate lors d'une nouvelle faille (type Log4Shell).
2. **Conformité CRA** — preuve de la maîtrise de la supply chain.
3. **Soutenabilité** — identification des dépendances critiques à soutenir.

CRA : trois rôles, trois régimes

Rôle	Qui ?	Obligations principales
Fabricant	Met un produit sur le marché (éditeur, intégrateur)	Marquage CE, SBOM, déclaration de conformité, gestion des vulnérabilités, support ≥ 5 ans
Distributeur	Revend le produit	Vérifier la conformité du fabricant, coopérer en cas d'incident
Open Source Steward	Personne morale qui soutient un projet OSS (fondation, éditeur contributeur)	Politique de cybersécurité documentée, coordination de la divulgation, signalement d'incidents graves

Nouveauté : le rôle d'**Open Source Steward** (« intendant » en français) est créé **spécifiquement** par le CRA — régime allégé par rapport au fabricant.

CRA : l'exception open source

Principe : seuls les produits distribués **dans un cadre commercial** sont soumis au CRA.

Critères d'activité non commerciale (hors champ du CRA) : distribution gratuite et sans but lucratif, financement par dons ou subventions, absence de services payants associés.

Stratégie de publication parallèle : l'appréciation se fait **par produit mis sur le marché**, pas par projet. Un même code peut donc donner lieu à deux régimes : la version OSS non commercialisée relève du rôle de *Steward*, la version commerciale (support, SaaS, distribution payante) relève du rôle de **Fabricant**.

Un projet **purement communautaire** (pas d'éditeur derrière) est **hors champ** du CRA. Mais dès qu'une entreprise le met sur le marché (même modifié), elle devient fabricant pour cette version.

Guide CNLL / inno³ : <https://code.inno3.eu/ouvert/guide-cra>.

CRA : ce que ça change pour les éditeurs

Transformations internes

1. Articuler politique open source et politique cyber — convergentes mais distinctes.
2. Connaître l'**intégralité** des composants intégrés (développements internes, sous-traitance, dépendances transitives).
3. Maintenir la conformité sur toute la durée de vie du produit (≥ 5 ans).

Transformations externes

- Clauses contractuelles : obtention des SBOM en amont, reversement *upstream* des correctifs.
- Passer d'une logique d'**usage** passif de l'OSS à un rôle **proactif de contribution** (cf. session 9).

▮ « *I am not your supplier.* » — **Thomas Depierre**, mainteneur OSS.

Il n'y a pas de relation contractuelle entre mainteneurs et utilisateurs. Le CRA force les fabricants à l'assumer — ou à contribuer aux projets dont ils dépendent.

Maturité open source des organisations

Niveau	Caractéristiques
OSS subi	Usage opportuniste, informatique « grise », pas de politique
OSS maîtrisé	Risques encadrés, validation des usages, SBOM générés
OSS décidé	Engagement stratégique, contributions <i>upstream</i> , enjeu métier et RH

Le CRA est un accélérateur : il force le passage du niveau 1 au niveau 2, et récompense le niveau 3 (mutualisation des coûts de cybersécurité via les communautés).

Souveraineté numérique

L'open source est un levier clé de **souveraineté numérique** — la capacité à maîtriser ses infrastructures et ses données.

Risques de dépendance : deux textes américains, aux champs distincts mais convergents, fragilisent la confidentialité des données confiées à des acteurs US.

- Le **CLOUD Act** (*Clarifying Lawful Overseas Use of Data Act*, 2018) : les autorités américaines peuvent contraindre une entreprise soumise au droit US à fournir les données qu'elle détient, **où qu'elles soient stockées dans le monde** — y compris dans une filiale européenne, et sans notification de l'utilisateur.
- **FISA Section 702** (*Foreign Intelligence Surveillance Act*, §702 ajouté en 2008, reconduit en 2024) : autorise la **surveillance de masse** — sans mandat individuel — des communications électroniques de personnes non-américaines, via les grands *electronic communications service providers*

américains (opérateurs cloud, messageries...). C’est la base juridique des programmes révélés par Snowden en 2013 (**PRISM**).

Ajouter à cela le *vendor lock-in* (dépendance à un éditeur unique, propriétaire ou *source-available*) et la perte de contrôle sur les mises à jour, la roadmap et les conditions d’usage.

Conséquence pour une organisation européenne : un simple hébergement chez un *hyperscaler* US, **même en datacenter européen**, expose potentiellement les données à ces deux régimes — un point régulièrement soulevé par la CNIL et l’EDPB, notamment dans les suites de l’arrêt **Schrems II** (2020) qui a invalidé le *Privacy Shield*.

L’open source comme réponse : code auditable et modifiable ; pas de dépendance à un éditeur unique ; hébergement sur une infrastructure souveraine possible ; conformité RGPD et NIS2 facilitée.

Réglementation européenne

Texte	Portée	Impact open source
RGPD (2018)	Données personnelles	Exige la maîtrise des données — favorise l’hébergement souverain
NIS2 (2024)	Sécurité des réseaux et SI	Obligations de gestion des risques supply chain
CRA (2027)	Cyber-résilience des produits	SBOM obligatoire, gestion des vulnérabilités
Sovereignty Package (annoncé mai 2026)	Paquet européen	Devrait inclure une stratégie « Open Source » renforcée

Outils de génération de SBOM

Outil	Formats	Langages
Syft (Anchore)	SPDX, CycloneDX	Multi
Trivy (Aqua)	SPDX, CycloneDX	Multi
cdxgen	CycloneDX	Multi
pip-audit	CycloneDX	Python
uv export –format cyclonedx1.5	CycloneDX	Python
npm audit	Propriétaire	JavaScript

Partie 4 — Gestion des vulnérabilités

Bases de données

Bases officielles : **CVE** (MITRE) — attribue les identifiants universels ; **NVD** (NIST) — fournit les scores CVSS et l’analyse ; **GHSA** (GitHub Security Advisories).

Bases spécialisées OSS : **OSV** (Google), Snyk Vulnerability DB, Sonatype OSS Index.

Alerte récente : en avril 2025, l’administration Trump (DOGE) a coupé les budgets de MITRE, menaçant le fonctionnement de la base CVE — illustration de la **dépendance** du monde entier à une infrastructure US. Cf. <https://www.iisf.ie/CVE-System-grinds-to-a-near-halt>.

Le score CVSS

Common Vulnerability Scoring System.

Score	Sévérité	Exemple
0.0	Aucune	—
0.1-3.9	Faible	Info disclosure mineur
4.0-6.9	Moyenne	DoS partiel
7.0-8.9	Haute	RCE avec conditions
9.0-10.0	Critique	RCE sans auth (Log4Shell)

Facteurs : vecteur d'attaque, complexité, privilèges requis, impact sur la confidentialité/intégrité/disponibilité.

Outils de scan (SCA — *Software Composition Analysis*)

- **Dependabot** (GitHub) — alertes + PRs de mise à jour automatiques.
- **Snyk, OWASP Dependency-Check, Trivy, Grype.**

Exemple de configuration Dependabot :

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule: { interval: "weekly" }
    open-pull-requests-limit: 10
```

Partie 5 — Bonnes pratiques

Le problème du mainteneur solitaire

« *Imaginez que toute l'infrastructure numérique mondiale dépend d'un projet maintenu par un type au hasard dans le Nebraska.* » — Paraphrase du XKCD #2347 (« Dependency »).

C'est exactement ce qui s'est passé avec Log4j (2 mainteneurs bénévoles), XZ Utils (1 mainteneur en *burnout*), OpenSSL avant Heartbleed (1 développeur à temps plein pour la lib TLS la plus utilisée au monde).

Conséquence : la sécurité de la supply chain a mis en lumière un problème structurel de **financement** et de **soutenabilité** de l'open source (cf. session 9).

Pour les développeurs

Gestion des dépendances : minimiser le nombre, vérifier que chacune est activement maintenue, verrouiller les versions avec des *lock files*, auditer régulièrement les dépendances installées.

Sécurité : activer Dependabot ou Snyk, mettre à jour rapidement les versions vulnérables, ne pas ignorer les alertes, vérifier soigneusement les nouvelles dépendances avant de les adopter.

Évaluer une dépendance

1. **Maintenance** — dernière release ? Issues traitées ?
2. **Communauté** — nombre de contributeurs ? *Bus factor* ?
3. **Sécurité** — CVE passées ? Temps de réponse ?
4. **Nécessité** — vraiment utile ? Alternative possible ?
5. **Licence** — compatible avec votre projet ?

Outils : **OpenSSF Scorecard**, Snyk Advisor, Libraries.io.

OpenSSF Scorecard

Évalue **automatiquement** la sécurité d'un projet.

Critère	Description
Binary-Artifacts	Pas de binaires dans le repo
Branch-Protection	Protection des branches
Code-Review	Reviews obligatoires
Dangerous-Workflow	Pas de workflows risqués
Maintained	Activité récente
Signed-Releases	Signatures cryptographiques
Vulnerabilities	Pas de vulnérabilités connues

Référence : <https://scorecard.dev/>.

OWASP — la communauté qui se prend en main

Open Worldwide Application Security Project — fondation à but non lucratif créée en 2001, dédiée à la sécurité des applications web.

Modèle d'organisation : communauté ouverte de bénévoles ; contenu et outils publiés sous licences libres ou Creative Commons ; financement par dons, adhésions, événements ; présence mondiale via des *chapters* locaux.

Publications phares : **OWASP Top 10** (les 10 risques web les plus courants — référence mondiale), **ASVS** (standard de vérification sécurité), **Cheat Sheets** (guides pratiques), **CycloneDX** (format SBOM).

Outils : **ZAP** (scanner web), Dependency-Check, DefectDojo, entre autres.

OWASP est un **contre-modèle vertueux** : une communauté qui se structure pour produire elle-même les ressources de sécurité que l'industrie seule ne produisait pas. À distinguer d'**OpenSSF** (Linux Foundation), qui se concentre sur la sécurité de l'open source lui-même (supply chain, Scorecard, SLSA).

Pour les organisations

Côté politique : registre de dépendances approuvées, processus de validation avant adoption, SLA de mise à jour, formation continue des développeurs.

Côté outillage : SCA (*Software Composition Analysis*) intégré dans la CI/CD, génération automatique de SBOM, monitoring continu, plan de réponse aux incidents.

Builds reproductibles

Principe : à partir du même code source, le build produit un résultat **bit-à-bit identique**, quel que soit l'environnement.

Pourquoi c'est important :

- Permet de **vérifier** qu'un binaire distribué correspond bien au code source.
- Détecte les altérations malveillantes dans le processus de build (cf. XZ Utils).
- Rend le *Trusting Trust* de Ken Thompson (1984) vérifiable.

Projet de référence : <https://reproducible-builds.org/> — Debian, Arch Linux, NixOS y travaillent activement.

Le framework SLSA

Supply-chain Levels for Software Artifacts — quatre niveaux de garantie d'intégrité :

Niveau	Exigences
SLSA 1	Documentation du processus de build
SLSA 2	Build hébergé, provenance signée
SLSA 3	Build isolé, provenance non-falsifiable
SLSA 4	Build hermétique, double vérification

Les cooldowns de dépendances

Idée simple : attendre quelques jours avant d'installer une nouvelle version d'un paquet.

Pourquoi ça marche : dans la plupart des attaques récentes (Ultralytics, Nx, rspack, Shai Hulud...), la fenêtre entre publication du code malveillant et détection par la communauté est < **1 semaine**. Un cooldown de 7 à 14 jours aurait bloqué la quasi-totalité de ces attaques.

Adoption massive en 2025-2026 :

- **pnpm, Yarn, Bun, uv** (Python), **npm** — tous ont introduit un paramètre `minimumReleaseAge` / `--exclude-newer`.
- **Cargo, Go, NuGet** : en cours.

Exemple (uv) : `uv add --exclude-newer=7d requests`.

À retenir : c'est gratuit, facile, et probablement la mitigation la plus efficace contre les attaques supply chain modernes.

Sources : William Woodruff — blog.yossarian.net ; Cal Paterson — calpaterson.com/deps.html.

Bonnes pratiques — l'exemple Astral

Astral (éditeur de `uv` et `ruff`, largement utilisés en Python) a publié son *playbook* sécurité. Quelques principes simples :

Protéger le code et le CI : authentification forte (2FA matérielle), droits minimaux par défaut, versions précises des outils utilisés dans le CI (pas de mise à jour automatique silencieuse).

Protéger les releases : supprimer les mots de passe et tokens *long-lived* ; signer chaque release avec une preuve vérifiable d'origine ; double approbation humaine avant toute publication.

Côté dépendances : minimiser, appliquer les cooldowns, contribuer aux projets amont dont on dépend, les financer si possible.

Source : <https://astral.sh/blog/open-source-security-at-astral>.

Partie 6 — IA et sécurité open source

Les LLM rebattent les cartes

Les modèles de langage transforment la recherche de vulnérabilités et la *supply chain* — pour le meilleur et pour le pire.

Opportunités : détection automatisée de bugs et vulnérabilités, génération de patchs, audit de code à grande échelle, aide aux mainteneurs pour le triage et la revue.

Risques nouveaux : *slop vulnerability reports* — signalements de masse souvent faux, qui saturent les mainteneurs ; ingestion non consentie du code par les modèles ; **slopsquatting** — dépendances « hallucinées » par les LLM, variante du *typosquatting* (étude Lasso Security : **20 %** des

paquets cités par LLM n'existent pas) ; agents autonomes exploitant à grande échelle la surface d'attaque.

Controverse cal.com (avril 2026)

Cal.com (alternative open source à Calendly) annonce une **fermeture partielle** de son code.

Motivations invoquées :

- Flot de faux rapports de vulnérabilités générés par LLM — temps mainteneur gaspillé.
- Concurrents utilisant les LLM pour cloner la base de code rapidement.
- Difficulté à monétiser un OSS totalement ouvert face à ces pressions.

Ce que ça illustre :

- Fragilité du contrat social de l'open source face à l'IA.
- Lien avec les tensions vues en session 9 (VC, exit, *bait-and-switch*) — peut-être un prétexte.
- Précédent inquiétant : d'autres projets pourraient suivre.

Source : <https://www.strix.ai/blog/cal-com-is-closing-its-code-due-to-ai-threats>.

Ce qu'il faut retenir

1. **Supply chain** : 90 %+ du code provient de dépendances open source.
2. **Incidents** : Log4Shell et XZ Utils montrent les risques — technique (RCE) et humain (social engineering).
3. **SBOM** : inventaire obligatoire (CRA, Executive Order) — **Sécurité + Conformité + Soutenabilité**.
4. **CRA** : trois rôles (**Fabricant, Distributeur, Open Source Steward**), exception OSS non commercial.
5. **Réglementation** : RGPD, NIS2 — en tension avec le CLOUD Act et le FISA américain ; l'open source comme contre-mesure et levier de souveraineté.
6. **IA** : nouveaux risques (slop reports, cal.com, slopsquatting) et nouvelles bonnes pratiques (Astral, cooldowns).
7. **Outils et pratiques** : Dependabot, Trivy, cooldowns, attestations, minimiser, contribuer *upstream*.

Le paradoxe de la sécurité open source

Avantages : code auditable, corrections rapides, communauté vigilante, transparence totale du fonctionnement.

Risques : mainteneurs non payés (et donc vulnérables au *burnout*, à l'ingénierie sociale, etc.) ; surface d'attaque visible par les attaquants ; confiance implicite par défaut envers les dépendances ; dépendances transitives difficiles à inventorier et à auditer.

■ « *Given enough eyeballs, all bugs are shallow* » — mais qui regarde ?

Pour aller plus loin

Réglementation

- Texte officiel CRA : <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>
- Guide CRA CNLL / inno³ : <https://code.inno3.eu/ouvert/guide-cra>

Standards et frameworks

- **SPDX** : <https://spdx.dev>
- **CycloneDX** : <https://cyclonedx.org>
- **OpenSSF Scorecard** : <https://scorecard.dev>

- **SLSA Framework** : <https://slsa.dev>
- **Reproducible Builds** : <https://reproducible-builds.org>

Communautés et fondations sécurité

- **OWASP** : <https://owasp.org>
- **OpenSSF** (Linux Foundation) : <https://openssf.org>

Financement de l'OSS

- **Sovereign Tech Fund** (Allemagne) : <https://www.sovereign.tech>
- **Open Source Pledge** : <https://opensourcepledge.com>